PROYECTO ALPHA: ¡Pégale al monstruo!

Evaluación experimental de desempeño

Luis Fernando Rodriguez Retama 208047
Mauricia Peña López Ostolaza 205688

Resumen del experimento
Método: 10 niveles de carga (N=50,100,...,500 clientes concurrentes)
Repeticiones por configuración: R=10
Métricas: tiempo promedio y desviación estándar de respuesta de registro y juego, % de éxito

1. Introducción

Proyecto Alpha es una implementación del juego de feria “pégale al topo” en un entorno distribuido. Tiene un servidor (ServidorMonstruos.java) y clientes (JugadorMonstruo.java), los cuales se registran y juegan enviando solicitudes para pegarle a los monstruos. La arquitectura combina dos canales de comunicación:

- **Canal JMS**: el servidor publica eventos de juego (INICIO, MONSTRUO:<casilla>, GANADOR:<jugador>) en el tópico PegaleAlMonstruo de un broker ActiveMQ. Todos los clientes suscritos (jugadores) reciben los eventos simultáneamente y se usa encapsulamiento en la publicación de los mensajes (sólo por seguridad).
- **Canal TCP**: las operaciones de control: registro de jugadores (puerto 33000) y envío de golpes (puerto 36900) sobre sockets TCP directos.

Donde los componentes principales son:

- **ServidorMonstruos**: gestiona el marcador en un hashmap concurrente, genera monstruos cada 2s en casillas aleatorias, acepta el primer golpe válido por monstruo y anuncia ganador al alcanzar 5 puntos.
- **EscuchadorLogin** (puerto 33000): registra jugadores mediante un ServerSocket TCP, desplegando una pantalla de login y llamando al servidor para registrarlo.
- **EscuchadorGolpes** (puerto 36900): mantiene conexiones TCP persistentes por cliente. Procesa múltiples golpes por conexión respondiendo HIT_OK o HIT_REJECTED y llama al servidor para hacer las actualizaciones de puntaje.

- **JugadorMonstruo**: cliente Swing que combina JMS para visualizar el tablero y TCP para enviar golpes sin recrear socket.
- **Estresador**: generador de carga sintética que simula hasta 500 clientes concurrentes y mide latencias de registro y golpe.

El objetivo de este informe, es evaluar el rendimiento del sistema y analizar cómo maneja un aumento en solicitudes durante el registro y el juego. Para esto se evaluaron las siguientes métricas:

- **Tiempo promedio de registro**: media del tiempo entre el envío del nombre y la recepción de la respuesta OK del registro.
- **Tiempo promedio de golpe**: Media del tiempo entre el envío del golpe por TCP y la recepción del ACK (HIT_OK o HIT_REJECTED).
- **Desviación estándar para los tiempos** (registro y golpes).
- **Porcentaje de éxito**: Proporción de logins TCP que reciben respuesta OK del servidor.

2. Definición del experimento

El sistema debe ser capaz de manejar un número creciente de jugadores concurrentes y soportar las siguientes tareas:

1. Registro de jugadores: Cada jugador se conecta al servidor, envía su nombre y recibe un mensaje cuando puede empezar el juego.

2. Enviar golpes: Cada jugador puede enviar golpes concurrentemente, y recibe el acuse de acierto.
Se establecieron los siguientes parámetros para el estresamiento del juego:

- Configuraciones de carga (N): 50,100,150,...,500 jugadores
- Número de repeticiones (R): para cada configuración
- Entorno de pruebas:
    - Software: Java openJDK 25
    - Hardware: Intel i7 gen 13
    - Servidor: ActiveMQ broker en localhost:61616
    - Clientes: Estresador ejecutado en la misma JVM que el servidor (local).
    - Concurrencia del servidor: cada conexión entrante genera un hilo independiente (EscuchadorLogin / EscuchadorGolpes)
    - Sistema operativo: JVM sobre plataforma local.

3. Protocolo del estresador

Para cada valor de N, el Estresador ejecuta R = 10 repeticiones independientes. Cada repetición comprende dos fases secuenciales:

1. **Fase A – Login concurrente**: se lanzan N hilos simultáneos. Cada hilo abre un socket TCP al puerto 33000, envía el nombre del jugador con writeUTF() y espera la respuesta. Se mide el tiempo entre t_0 (apertura del socket) y t_1 (lectura de confirmación). Se registra éxito si la respuesta comienza con «OK».
2. **Fase B – Primer golpe concurrente**: solo para los jugadores que conectaron exitosamente en la Fase A, se lanzan un golpe al servidor. Cada jugador tendrá una conexión TCP al puerto 36900 para este propósito. Se mide desde el envío hasta la recepción del ack.

Las dos fases son secuenciales dentro de cada repetición (se espera join() de todos los hilos de login antes de iniciar los golpes).

4. Resultados

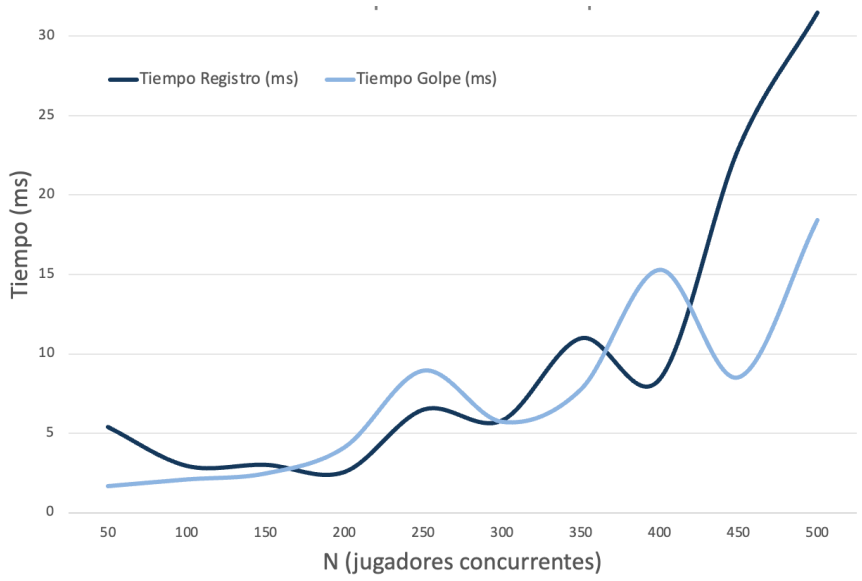
La Tabla 1 presenta las cinco métricas calculadas para cada nivel de carga (N). Cada fila resume 10 repeticiones × N clientes, es decir, entre 500 y 5,000 mediciones individuales por celda.

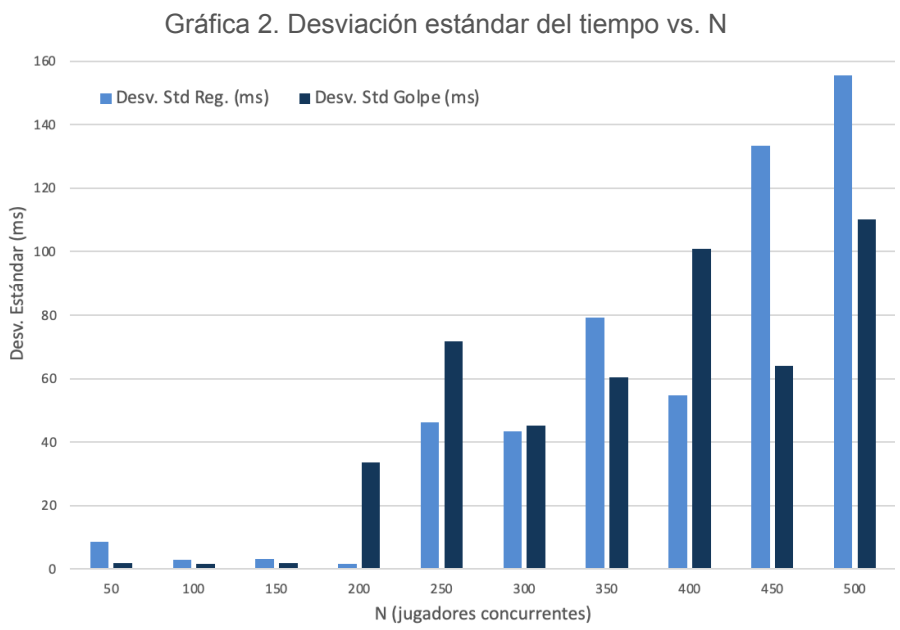
N	Éxito (%)	μ Reg. (ms)	Σ Reg. (ms)	μ Conex. (ms)	μ Reg. (ms)
50	100%	5.385	8.593	1.687565	1.859737
100	100%	2.938	2.921	2.109802	1.52277
150	100%	3.007	3.100	2.48461	1.882054
200	100%	2.553	1.716	4.128242	33.485263
250	100%	6.478	46.327	8.955715	71.718173
300	100%	5.79365	43.265661	5.734568	45.107689
350	100%	10.967612	79.196873	7.77585	60.435667
400	100%	8.396109	54.635952	15.300068	100.983705
450	100%	22.924567	133.414329	8.534374	63.954264
500	100%	31.461298	155.406667	18.428415	110.19742

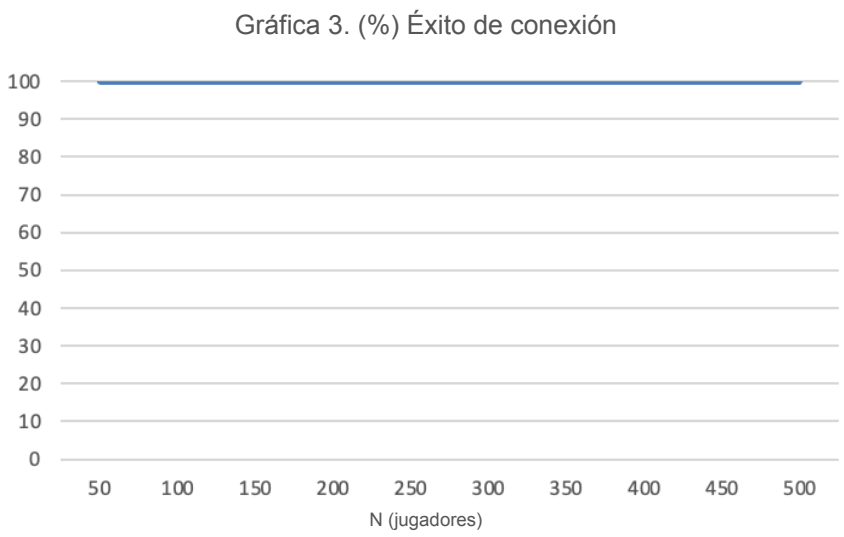
Tabla 1. Métricas agregadas por nivel de carga (N = clientes concurrentes, R = 10 repeticiones).

5. Gráficas

Gráfica 1.Tiempo promedio de respuesta vs. N







6. Análisis e interpretación

Primero analizaremos el tiempo y éxito de los registros. El porcentaje de éxito de conexión se mantuvo en 100 % para todos los valores de N evaluados. Este resultado indica que el sistema no rechazó ni descartó ninguna solicitud de registro durante la prueba. Sin embargo, hubo cambios en la latencia. A partir de N = 250 la variación estándar del tiempo de conexión aumentó significativamente. Para N ≤ 200, el tiempo promedio de conexión estuvo entre 2.6 ms y 5.4 ms, con desviaciones estándar menores a 9 ms. Podemos considerar que esta carga es estable porque el servidor atiende cada solicitud de registro en pocos milisegundos consistentemente.

Sin embargo, a partir de N = 250, el tiempo promedio sube ligeramente a 6.5 ms, pero la desviación estándar se dispara a 46.3 ms. Esto nos indica que una fracción de los clientes experimenta latencias mucho mayores al promedio. Finalmente, observamos que en N = 500, la media se dispara a

31.5 ms y la desviación estándar 155.4 ms, por lo que es una carga que retarda el servicio en general. En esta implementación, una mayor concurrencia implica una mayor latencia. Esto sugiere que el servidor aunque absorbe todas las conexiones, las cargas altas lo alentan.

Para el caso del tiempo promedio para golpe, hay variaciones aún mayores. Si observamos la gráfica 3/tabla 1, notamos que hay varios picos, en N = 200 (media 4.1 ms, desv. 33.5 ms), N = 400 (media 15.3 ms, desv. 101.0 ms) y N = 500 (media 18.4 ms, desv. 110.2 ms). Para N = 300, la media (5.7 ms) es incluso menor que para N = 400. El posible cuello de botella es el método synchronized tryHit(), ya que cuando hay carga alta, los N hilos para los golpes se serializan, por lo que el primer hilo responde en 0-2ms y los demás se acumulan en cola, lo que explica las altas desviaciones estándar. Además, como en las pruebas la mayoría de los golpes llegan cuando no hay monstruo activo, el tiempo medido corresponde al viaje de ida y vuelta por la red sin contar la actualización de puntuaciones en el servidor.

7. Conclusión

El sistema es funcional y robusto bajo cargas bajas (50-200 jugadores). Sin embargo, bajo cargas más altas (≥400 jugadores), se observó una mayor latencia y variabilidad en los tiempos de registro y golpes. Sin embargo, se mantuvo un porcentaje de 100% en las conexiones exitosas. Para mejorar estas estadísticas, se debería optimizar el manejo de conexiones TCP en el servidor. Una manera de mejorarlo sería escalar horizontalmente el servidor utilizando múltiples instancias del ServidorMonstruos. También se podría experimentar con balanceadores de carga o pausas para reducir problemas de saturación.